MINDSPEED™
BUILD IT FIRST

Application Note
For
Description of
C2000 HIF Driver Tx resource recycling
Issue and Fix
Bug #78363

Contents

1. Description of Linux network TX operation

1.1. *HIF TX operation*

When a network interface driver passes a packet to the hardware device for transmission, it generally needs to be notified by the device that the packet was sent, so that the various data buffers and structures that held the packet can be recovered and reused by the software. One common way to generate such a notification is for the device to trigger an interrupt once a packet has been transmitted. The HIF hardware block present in the C2000 PFE offers such a feature, but for performance reasons a batching scheme was used instead: the software keeps track of packets that have been sent to the device but not recycled yet, and starts recycling buffers and descriptors once the number of descriptors pending recycling exceeds a given threshold. This offers 2 main advantages compared to an interrupt-based scheme:

- TX interrupts are virtually eliminated, greatly reducing the overhead of per-packet interrupt handling and context switching,
- Recycling packets in batches means the recycling code and data are likely to be in cache after the first iteration, which improves the CPU efficiency when executing that code.

The recycling code is mostly called from the packet TX path: when a packet is sent, the driver checks for the number of pending descriptors, and if it is above a threshold (32 by default), it looks through the HIF TX ring buffer for packets ready to be recycled, until the number of pending descriptors goes back below the threshold. If the number of pending descriptors is below the threshold, nothing is done. To avoid tying up resources for a long time, a timer is also triggered periodically (every 500ms) to release any packet than can be even if there are less than threshold descriptors pending.

1.2. *TSO case*

The TSO implementation on the C2000 actually splits packets at the driver-level. For each chunk of data, the low-level headers are added, and as a result for each TSO packet the HIF will receive a series of smaller packets, each consisting of at least 2 descriptors (one pointing to the low-level headers, the other to the TCP data). Since the threshold described above refers to the number of HIF descriptors, this means recycling of packets will be done pretty much every time a full TSO packet is sent.

1.3. *Linux send buffer size*

Under most OSes, network sockets have a notion of send buffer size: it refers to the amount of data (in bytes) that has already been sent over the socket by the application but is still being held up somewhere in the system, waiting to be pushed out to the network. Under Linux and the C2000, this includes packets that were transmitted by the HIF but not released by the device yet. Furthermore, what is

accounted for per packet is the whole memory consumed, i.e. the __whole__ packet buffer and any data structure related to it, such as the skb.

The maximum send buffer size is configurable at the socket-level by the application, and is used by Linux to determine whether to accept new data from the application or not: if the current send buffer size exceeds the maximum value, the data will be rejected by Linux, and the “send” system call will return an error, which should be used by the application as a hint to retry sending later.

2. Problem description

The problem can arise when the threshold at which the driver starts recycling packets represents a bigger amount of data than the maximum send buffer size of an existing socket.

The below explanation assumes the following values:

- Threshold for recycling descriptors: 32
- Timer interval for flushing queues of any pending packet: 500ms
- Amount of memory consumed per-packet: 2240 (based on actual system measurements; this may vary a bit depending on the actual skb structure size, which can itself vary depending on the networking options configured in the Linux kernel)

Based on the previous description of HIF TX operation, packets will be released by the driver:

- when at least 32 descriptors are pending (transmitted but not released yet), which represents a maximum of 71680 bytes (32*2240),
- every 500ms.

If the send buffer size is set to say 32kB, then the application will have to stop sending packets once 32kB are "pending". This is less than 71680 bytes, so the driver will not try to free any packet, and the application will have to wait about 500ms for space to be available again in the send buffer.

The maximum throughput for such a socket would then be around 1514*32*2 = 95kB/s (32 packets will be flushed every 500ms so twice per second, and each packet can be up to 1514 bytes at the Ethernet level).

If however the send buffer size is set to 256kB, then the application will have to stop only once 256kB are "pending". Before this happens however, the driver will reach a state where more than 32 descriptors are pending, and it will then start releasing packets (at the rate of one for every TX packet). The send buffer will at that point start emptying at about the same rate it fills up, so the application will never have to stop because the send buffer is full.

In practice, the problem is not visible with TCP sockets, because:

- TCP sockets usually have a socket send size larger than 71680 to ensure good throughput over links with significant latencies,
- On the C2000, TSO is enabled by default, and as described before this means packet recycling will be done pretty much each time a TSO packet is transmitted.

The problem can however easily be experienced with UDP sockets using a small send buffer size of 32kB.

3. Description of changes

The decision to start batch recycling was enhanced with additional criteria:

- We now take into account the size of the send buffer for the socket the current packet comes from, and the driver starts batch recycling if that send buffer is half full.

In addition, several smaller improvements were also made, to:

- Improve the code efficiency by limiting the number of function calls, and make sure recycling is done in large batches,
- Avoid flushing a queue in the timer code if traffic was seen in that queue since the last timeout (this prevents locking the queue when an application may want to transmit).